

Architecture Concept & Implementation Report

Scalable BERTopic Modeling on 100 Million Documents with a Limited Training Subset

1. Introduction & Objectives

The primary objective of this project is to implement a highly scalable topic modeling system using **BERTopic** on a massive dataset consisting of **100 million documents**. Due to hardware constraints and processing time optimization, the direct training phase is strictly limited to a maximum of **5 million documents**. Despite this limitation, the final model is expected to achieve a comprehensive semantic coverage capturing at least **97% of all existing topics** across the entire 100-million-document dataset.

To realize this ambitious goal, the architecture utilizes a dedicated 5-phase framework. This approach leverages the distributed computing power of an **Apache Spark** cluster for pre-processing and large-scale sampling, combined with the ultra-performant **Polars** library and **RAPIDS cuML (GPU)** for the core training and efficient streaming inference.

Core Challenge: Protecting the "Long Tail"

Standard random sampling of 5% of the data would inevitably overlook rare niche topics. The concept therefore relies on *Semantic Diversity Sampling*, which pre-structures the vector space to intentionally extract representative documents from every single semantic zone.

2. Phase 1: Intelligent Sampling (Semantic Diversity Sampling)

Technology Stack: PySpark, Spark MLlib, Apache Arrow, Polars

In this phase, the 100 million documents are analyzed via Spark. An initial coarse subset of 20 million documents is drawn, vectorized using light-weight feature hashing (TF), and clustered via Mini-Batch K-Means into 10,000 distinct semantic groups. A window function is then applied to extract an equal number of documents from each cluster. This guarantees representation from all semantic regions, including microscopic niches. The resulting 5 million documents are transferred with zero-copy efficiency to a Polars DataFrame on the local GPU node via Apache Arrow.

Phase 1 Implementation:

```
import polars as pl
import pyarrow as pa
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, rand, row_number
from pyspark.sql.window import Window
from pyspark.ml.feature import Tokenizer, HashingTF
from pyspark.ml.clustering import KMeans
from pyspark.ml import Pipeline

def semantic_diversity_sampling_to_polars(
    spark: SparkSession,
    df_100m,
    text_col: str = "text",
    total_sample_size: int = 5_000_000,
    intermediate_sample_size: int = 20_000_000,
    n_clusters: int = 10_000,
    seed: int = 42
) -> pl.DataFrame:
    print("Step 1: Draw initial coarse subset (20M documents)...")
    fraction = intermediate_sample_size / 100_000_000
    df_20m = df_100m.sample(withReplacement=False, fraction=fraction, seed=seed)

    print("Step 2: Prepare lightweight tokenization and vectorization...")
    tokenizer = Tokenizer(inputCol=text_col, outputCol="words")
    hashing_tf = HashingTF(inputCol="words", outputCol="features", numFeatures=16384)

    print(f"Step 3: Compute K-Means clustering into {n_clusters} clusters...")
    kmeans = KMeans(featuresCol="features", predictionCol="cluster_id", k=n_clusters, seed=seed, m

    pipeline = Pipeline(stages=[tokenizer, hashing_tf, kmeans])
    model = pipeline.fit(df_20m)
    df_clustered = model.transform(df_20m)

    print("Step 4: Execute stratified sampling across clusters...")
    docs_per_cluster = max(1, total_sample_size // n_clusters)
    window_spec = Window.partitionBy("cluster_id").orderBy(rand(seed))

    df_sampled = (
        df_clustered
        .withColumn("row_num", row_number().over(window_spec))
        .filter(col("row_num") <= docs_per_cluster)
        .select("id_column", text_col, "cluster_id")
    )

    print("Step 5: High-speed data transfer from Spark to Polars via Apache Arrow...")
    spark.conf.set("spark.sql.execution.arrow.pyspark.enabled", "true")

    pandas_df = df_sampled.toPandas()
    polars_df = pl.from_pandas(pandas_df)

    print(f"Success! Output dataset contains {polars_df.height} rows.")
    return polars_df
```

3. Phase 2: Hardware & Infrastructure Setup

Processing 5 million high-dimensional dense embeddings strictly requires dedicated graphics processing units (GPUs). Running CPU-bound implementations of UMAP and HDBSCAN on millions of documents would trigger Out-of-Memory (OOM) exceptions or require weeks of execution time.

- **Hardware Recommendation:** Minimum of 1x NVIDIA A100 (80GB VRAM) or equivalent enterprise-grade hardware accelerator. Alternatively, enthusiast consumer cards like the RTX 4090 (24GB VRAM) can be utilized by lowering the embedding batch size.
- **Software Stack:** Mandatory inclusion of RAPIDS cuML. BERTopic integrates out-of-the-box with cuML components, offloading geometrical dimensionality reduction (UMAP) and density-based clustering (HDBSCAN) straight to the GPU VRAM. This reduces the core execution time down to hours.

4. Phase 3: The BERTopic Training Pipeline (on 5M Documents)

Technology Stack: Polars, SentenceTransformers, BERTopic, RAPIDS cuML (GPU)

To execute stable training on the 5 million isolated documents, a sub-model training paradigm is introduced. The heavy geometric fitting of UMAP and HDBSCAN runs on a core slice of 1 million documents. The remaining 4 million documents within the training pool are immediately projected into this space using the trained components (Inference).

A vital setting here is configuring `prediction_data=True` inside the HDBSCAN parameters, which is mandatory for enabling downstream inferences. Following an embedding-based noise reduction strategy (Outlier Reduction), the final c-TF-IDF keyword distributions are updated across the entirety of the 5-million-document training base.

Phase 3 Implementation:

```

import polars as pl
import numpy as np
from sentence_transformers import SentenceTransformer
from bertopic import BERTopic
from cuml.manifold import UMAP
from cuml.cluster import HDBSCAN

def train_bertopic_large_scale(
    df: pl.DataFrame,
    text_col: str = "text",
    train_size: int = 1_000_000,
    model_name: str = "paraphrase-multilingual-MiniLM-L12-v2",
    save_path: str = "./bertopic_5m_model"
):
    print("Step 1: Extract string sequences from Polars...")
    docs = df[text_col].to_list()
    train_docs = docs[:train_size]
    infer_docs = docs[train_size:]

    print(f"Step 2: Generate dense text embeddings for the {train_size} core subset...")
    embedding_model = SentenceTransformer(model_name)
    train_embeddings = embedding_model.encode(train_docs, show_progress_bar=True, batch_size=512)

    print("Step 3: Instantiate GPU-accelerated cuML UMAP and HDBSCAN...")
    umap_model = UMAP(n_components=5, n_neighbors=15, min_dist=0.0)
    hdbscan_model = HDBSCAN(min_samples=50, min_cluster_size=500, prediction_data=True)

    topic_model = BERTopic(
        umap_model=umap_model,
        hdbscan_model=hdbscan_model,
        language="multilingual",
        calculate_probabilities=False,
        verbose=True
    )

    print(f"Step 4: Train core BERTopic model on {train_size} documents...")
    train_topics, _ = topic_model.fit_transform(train_docs, train_embeddings)

    print(f"Step 5: Infer categories for the remaining {len(infer_docs)} training records...")
    infer_embeddings = embedding_model.encode(infer_docs, show_progress_bar=True, batch_size=512)
    infer_topics, _ = topic_model.transform(infer_docs, infer_embeddings)

    print("Step 6: Concatenate all 5 million target records...")
    all_docs = train_docs + infer_docs
    all_topics = train_topics + infer_topics
    all_embeddings = np.vstack((train_embeddings, infer_embeddings))

    print("Step 7: Perform Outlier Reduction (Minimize Topic -1)...")
    new_topics = topic_model.reduce_outliers(
        all_docs, all_topics, strategy="embeddings", embeddings=all_embeddings
    )

    print("Step 8: Re-calculate c-TF-IDF matrix using all 5 million records...")
    topic_model.update_topics(all_docs, topics=new_topics)

    print(f"Step 9: Persist model using serialization at {save_path}...")
    topic_model.save(save_path, serialization="pickle")

```

```
print("Training phase fully completed!")  
return topic_model, new_topics
```

5. Phase 4: Inference on the Remaining 95M Documents

Technology Stack: PyArrow (Streaming), Polars, PyTorch VRAM Memory Management

Classifying the remaining 95 million un-trained records must strictly happen via data streaming batches. To avoid RAM exhaustion, `pyarrow.parquet` streams localized chunks (e.g., 500,000 lines) straight from storage. These chunks move sequentially through the sentence embedder and the BERTopic transformation pipeline. The resulting vectors are instantly persisted using a `ParquetWriter` to enable an append-only pipeline. Crucially, the python Garbage Collector and PyTorch's CUDA cache management are triggered manually after every iteration.

Phase 4 Implementation:

```

import polars as pl
import pyarrow.parquet as pq
import pyarrow as pa
import gc
import torch
import os
from bertopic import BERTopic
from sentence_transformers import SentenceTransformer

def infer_bertopic_large_scale(
    input_parquet_path: str,
    output_parquet_path: str,
    model_path: str = "./bertopic_5m_model",
    embedding_model_name: str = "paraphrase-multilingual-MiniLM-L12-v2",
    text_col: str = "text",
    id_col: str = "id_column",
    chunk_size: int = 500_000
):
    print("Step 1: Load foundational models into GPU VRAM...")
    embedding_model = SentenceTransformer(embedding_model_name)
    topic_model = BERTopic.load(model_path)

    print(f"Step 2: Initialize I/O streaming handle for {input_parquet_path}...")
    parquet_file = pq.ParquetFile(input_parquet_path)

    if os.path.exists(output_parquet_path):
        os.remove(output_parquet_path)

    chunk_counter = 1
    total_processed = 0

    for batch in parquet_file.iter_batches(batch_size=chunk_size):
        print(f"\n--- Processing Batch Chunk {chunk_counter} ({batch.num_rows} Docs) ---")
        df_chunk = pl.from_arrow(batch)
        docs = df_chunk[text_col].to_list()
        doc_ids = df_chunk[id_col].to_list()

        print("    -> Generating embeddings...")
        embeddings = embedding_model.encode(docs, show_progress_bar=True, batch_size=512)

        print("    -> Executing BERTopic classification...")
        topics, _ = topic_model.transform(docs, embeddings)

        print("    -> Committing batch outputs to disk...")
        df_results = pl.DataFrame({id_col: doc_ids, "topic_id": topics})
        table = df_results.to_arrow()

        if chunk_counter == 1:
            writer = pq.ParquetWriter(output_parquet_path, table.schema)
            writer.write_table(table)
        else:
            writer.write_table(table)

        total_processed += batch.num_rows
        del df_chunk, docs, doc_ids, embeddings, topics, df_results, table
        gc.collect()
        torch.cuda.empty_cache()

        print(f"    -> Batch {chunk_counter} processed successfully. Total: {total_processed}")
        chunk_counter += 1

```



```
if 'writer' in locals():  
    writer.close()  
print(f"\n✅ Pipeline inference completed! {total_processed} records mapped.")
```

6. Phase 5: Validation of 97% Coverage (Diagnostic Analysis)

Technology Stack: Polars Lazy API (Out-of-Core Processing), Diagnostic BERTopic Fitting

To mathematically prove that the target 97% semantic coverage was met, a rigorous audit of the unassigned records (Topic -1) must be performed. Utilizing Polars Lazy API, a high-performance out-of-core streaming join combines the inference predictions back with the original source files. From these outliers, a diagnostic slice of 1 million documents is isolated and subjected to an independent, isolated BERTopic setup configuring a deliberately high cluster floor threshold (e.g., 1,000 documents).

Evaluation Metrics & Interpretation:

Scenario A (Success Condition): The diagnostic model produces structural noise (Topic -1) or microscopic boilerplate text groupings (e.g., footers, cookie statements). This confirms no macroeconomic themes were skipped during sampling.

Scenario B (Feedback Action Required): The evaluation identifies well-structured, distinct macro-clusters. This indicates a gap in Phase 1 sampling, requiring those specific document vectors to be injected back into the primary training pool.

Phase 5 Implementation:

```
import polars as pl
from sentence_transformers import SentenceTransformer
from bertopic import BERTopic
from cuml.manifold import UMAP
from cuml.cluster import HDBSCAN

def analyze_outliers_validation(
    inferred_results_path: str,
    original_texts_path: str,
    text_col: str = "text",
    id_col: str = "id_column",
    sample_size: int = 1_000_000
):
    print("Step 1: Scan and filter unmapped outliers (Topic -1) via Polars Lazy API...")
    df_inferred = pl.scan_parquet(inferred_results_path)
    df_original = pl.scan_parquet(original_texts_path)

    outliers_lazy = df_inferred.filter(pl.col("topic_id") == -1)
    joined_lazy = outliers_lazy.join(
        df_original.select([id_col, text_col]), on=id_col, how="inner"
    )

    print(f"Step 2: Collect data stream and draw a subset of {sample_size} records...")
    try:
        df_outliers = joined_lazy.collect(streaming=True)
    except Exception as e:
        print(f"Streaming failed, switching to standard execution: {e}")
        df_outliers = joined_lazy.collect()

    if df_outliers.height > sample_size:
        df_outliers = df_outliers.sample(n=sample_size, with_replacement=False, seed=42)

    outlier_docs = df_outliers[text_col].to_list()
    print(f"-> Loaded {len(outlier_docs)} outlier text entries.")

    print("Step 3: Fit an isolated diagnostic BERTopic model on the unmapped subset...")
    embedding_model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
    embeddings = embedding_model.encode(outlier_docs, show_progress_bar=True, batch_size=512)

    umap_model = UMAP(n_components=5, n_neighbors=15, min_dist=0.0)
    hdbscan_model = HDBSCAN(min_samples=50, min_cluster_size=1000)

    diagnostic_model = BERTopic(
        umap_model=umap_model,
        hdbscan_model=hdbscan_model,
        language="multilingual",
        verbose=True
    )

    topics, _ = diagnostic_model.fit_transform(outlier_docs, embeddings)

    print("\n=== Diagnostic Audit: Top 10 Found Clusters in Outlier Noise ===")
    topic_info = diagnostic_model.get_topic_info()
    print(topic_info.head(10))

    return diagnostic_model, topic_info
```

7. Summary & Structural Best Practices

The architectural framework described successfully couples distributed analytics pipelines (Spark) with ultra-fast localized multi-threaded in-memory engines (Polars & RAPIDS cuML). By abstracting core mathematical fitting (1M records), training-pool expansion (4M records), and block-buffered disk-streaming inference (95M records), this strategy bypasses traditional VRAM constraints. It provides an elegant, resource-efficient, and enterprise-grade solution to large-scale modern natural language processing initiatives.